2024国庆题目选讲-03
动态规划专题

Author:George_Plover

2024-10-03

NOI大纲：提高组

动态规划

- 【6】树型动态规划
- 【7】状态压缩动态规划
- 【8】动态规划的常用优化

Problem 1：【Gym103446G】Edge Groups

题意

给出一个具有 n 个结点的树，保证 n 是奇数。要求把 $n-1$ 条边两两分组，分成 $\frac{n-1}{2}$ 组。同一组的边必须有一个公共顶点。

问有多少种不同的分组方案？答案对 998244353 取模。

两个方案不同，当且仅当存在一条边，他在方案A中同组的边与他在方案B中同组的边不同。

$n \leq 10^5$

提示

树形DP

Problem 1：【Gym103446G】Edge Groups

思路

- 定义 $dp(i)$ 表示子树 i 的匹配方案数。注意到，如果 i 为根的子树中边的数量为奇数，那么要完美匹配必须把 i 的父亲边给用上。反之，则必须不用上父亲边实现子树内的完美匹配。
- 考虑转移方程：在已知结点 u 的子结点 $v_1, v_2, \cdots, v_k$ 的 dp 值的情况下，我们考虑计算 $dp(u)$ 。我们不妨假设这些子结点中有 r 个还保留了父亲边（即它们的子树中边数为偶数个），分别是 $v_1, v_2, \cdots, v_r$ 。那么 $dp(u)$ 的数量，除了要累乘 $dp(v_i), 1 \leq i \leq k$ ，还需要处理剩余的 r 条边的匹配。
- 因此接下来考虑一个问题（$f(r)$）：对于 r 条边，两两分组（如果是奇数条，则最后一个自成一组），有多少种不同的方案？容易分奇偶讨论利用递推式推出结论：

$$f(r) = \begin{cases} 1 \times 3 \times 5 \cdots \times r-2 \times r & ,r\text{是奇数} \\ f(r-1) & ,r\text{是偶数} \end{cases}$$

- 因此，线性预处理 $f(r)$ 之后，做 dp 即可，转移方程为 $dp(u) = f(r) \cdot \prod_{i=1}^{k} dp(v_i)$ 其中 r 表示 u 的子结点中保留的父亲边数量，k 表示 u 的子结点数量。特别的，叶子结点 $dp(u) = 1$ 。时间复杂度 $O(n)$

Problem 2: 【Gym103447G】Damaged Bicycle

题意

给出一张具有 n 个结点的，m 条边的带边权无向图。边权表示路径长度。

主人公从 1 号结点出发，目标是到 n 号结点。主人公步行的速度是 t 个单位距离每秒，骑车的速度是 r 个单位距离每秒。

在图中，有 $k(k \leq 18)$ 个共享单车，第 i 个单车位于结点 a_i 。每个单车有 p_i 的概率是损坏的，但只有当主人公走到单车所在结点并扫码后才会知道它是否损坏。

一旦主人公成功骑上一辆无损的共享单车，他将直接骑车到达终点。

问主人公在最优策略的情况下，期望最快多少秒能到达终点？

数据范围

$1 \leq n, m \leq 10^5, 0 \leq k \leq 18$ 。时间限制 3 s。

提示

主人公一旦骑上一辆共享单车，他将直接走最短路骑车到达终点。
策略指的是，主人公在任何一个已知状态下作的决策。比如已经扫了若干个车是坏的了，是直接走到终点去，还是继续尝试去扫下一辆车？

Problem 2：【Gym103447G】Damaged Bicycle

思路

- 考虑状压dp的状态： $dp(S,x)$ 表示已经扫了集合 S 里的共享单车，但全部都是坏的，且主人公目前位于结点 x 的情况下，此刻最优决策下主人公还需要花的时间的期望。注意到，x 只可能是 S 集合中的一个点，所以 x 的取值范围只有 k 个。
- 转移方程推理：主人公处于 S,x 的状态下，他有以下选择：

1. 直接走到终点，此时期望时间为 $dis(x,n)/t$ 。
2. 去找一辆不属于集合 S 的共享单车 j 扫一扫，这需要先花费 $dis(x,a_j)/t$ 的时间走过去。如果扫到的是坏的，则进入状态 $dp(S\cup j,a_j)$，该情况概率是 p_j 。否则，直接骑车去，此时期望时间为 $dis(a_j,n)/r$ ，该情况的概率是 $1-p_j$。

上述的 $dis(x,y)$表示x$ 到 y 的最短路。综上，我们的策略是，决定选择直接走去终点，还是选择找一个单车去扫。我们直接比较这些方案的期望时间即可。得到转移方程：

$$dp(S,x)=\min\left\{dis(x,n)/t,\ \min_{j\notin S}\left\{dis(x,a_j)/t+p_j\cdot dp(S\cup j,a_j)+(1-p_j)\cdot dis(a_j,n)/r\right\}\right\}$$

Problem 2：【Gym103447G】Damaged Bicycle

实现

注意到，虽然上面的表达式用到了两点间最短路，但可以发现起点和终点只可能是 k 辆车所在的点和起点和终点，因此可以分别以这些点为起点跑一次单源最短路预处理出来，用 $Dijkstra$ 算法即可，复杂度 $O(k(n+m)\log m)$。

之后，根据 S 集合从大到小的顺序 dp 即可，起始边界为 $dp(111\cdots1,a_j),j=1,2,\cdots,k$。 它们表示已经扫完所有车都是损坏的。因此这些状态下还需要的期望时间只能是 $dis(a_j,n)/t$ 。

最后，我们只需要对 $dp(000\cdots0,1)$ 状态输出即可。

总复杂度 $O(k(n+m)\log m+k^2\cdot 2^k)$

Problem 3：【Gym104081I】宠物对战

题意

现在有两类字符串，A 类和 B 类，A 类字符串有 n 个，B 类字符串有 m 个，每个字符串仅由小写字母组成。用这两类字符串交替拼接可以形成一个新的字符串，形如 abababab……和 bababababa……，其中 a表示 A 类中的某个字符串，b 表示 B 类中的某个字符串。注意，同一个字符串可以被重复利用。

现在有一个字符串 S 想问问你，S 是否是由 A、B 两类字符串交替拼接而成（即是否是上述的两种形式之一）。如果是，请输出 S 最少由多少个字符串拼接而成；如果不是，请输出 -1 。

数据范围： A 类字符串的长度之和不超过 5×10^5；B 类字符串的长度之和不超过 5×10^5。$|S| \leq 5000$。

提示

注意到 $|S|$ 很小，有没有 $O(|S|^2)$ 的算法？

Problem 3：【Gym104081I】宠物对战

思路

- 定义 $dp(x,0/1)$ 表示字符串 $S[x,|S|]$ （从下标 x 开始的后缀）最少能够被多少个串拼接而成，其中第二维度， 0 表示从 A 类字符串开始拼，1 表示从 B 类字符串开始拼。如果 dp 值为 -1 则表示这种方案不存在。
- 转移方程：

$$dp(x,0)=\min_{x<y\le|S|,S[x:y-1]\in A}\{dp(y,1)+1\}$$

$$dp(x,1)=\min_{x<y\le|S|,S[x:y-1]\in B}\{dp(y,0)+1\}$$

此外，需要注意特判 -1 非法值，不要用 -1 进行转移运算。

实现

上述 dp 方程的问题在于如何在枚举 y 的时候 $O(1)$ 判断 $S[x:y-1]$ 是不是 A 字符串集合里的字符串。这可以用字典树预处理 A 字符串集合实现。对字符串 B 同理。

总复杂度：$O(|S|^2+26(\sum|a_i|+\sum|b_i|))$

Problem 4: 【Gym103446I】Steadily Growing Steam

题意

一个卡牌游戏，给出 n 张卡牌，每个卡牌具有点数 p_i，具有价值 v_i 。游戏开始时，玩家可以选择至多 k 张牌，把这些牌的点数翻倍。然后他需要做下面的操作：

从 n 张牌里面，选出两个互不相交的卡牌集合 S 和 T ，使得 S 集合中的牌的点数之和与 T 集合的牌的点数之和相等。注意，不需要保证 $S \cup T$ 等于全集。也就是说有些卡可以不选进任何一个集合。然后玩家可以获得 S 和 T 集合中所有牌的价值之和。

请你输出玩家能获得的最大价值。

$n \leq 100, 0 \leq k \leq n, 1 \leq p_i \leq 13, |v_i| \leq 10^9$

提示

注意到 p_i 很小，并且 v_i 可以是负数。

Problem 4: 【Gym103446I】Steadily Growing Steam

思路

- 平衡分组问题：抽象成类似背包的dp问题，每个卡牌可以考虑装进 S 还是装进 T 。如果装进 S 则占用 p_i 的空间，如果装进 T 则占用 $-p_i$ 的空间。最后我们需要正好空间为 0 的状态。
- 定义 $dp(i,j,W)$ 表示考虑前 i 张卡牌，并且目前已经让 j 张牌点数翻倍，并且目前的空间占用为 W 的状态下，所选牌的最大价值和。
- 转移方程：考虑从 i 到 $i+1$ ，则是考虑第 $i+1$ 张牌，一共有 5 种操作：1. 不选这张牌。2. 选这张牌，共四种情况，即：对其（使用/不使用）边数翻倍的情况下，把这张牌装进（S/T）集合。于是对应的转移方程分别为：

$$dp(i+1,j,W) = dp(i,j,W)$$

$$dp(i+1,j+1,W+2p_i) = \max\left\{dp(i,j,W)+v_{i+1}, dp(i+1,j+1,W+2p_i)\right\}$$

$$dp(i+1,j,W+p_i) = \max\left\{dp(i,j,W)+v_{i+1}, dp(i+1,j,W+p_i)\right\}$$

$$dp(i+1,j+1,W-2p_i) = \max\left\{dp(i,j,W)+v_{i+1}, dp(i+1,j+1,W-2p_i)\right\}$$

$$dp(i+1,j,W-p_i) = \max\left\{dp(i,j,W)+v_{i+1}, dp(i+1,j,W-p_i)\right\}$$

Problem 4: 【Gym103446I】Steadily Growing Steam

实现

实现细节的一些小技巧：

W 维度需要支持负数值，可以用下标偏移来实现。
初始化值设置为负无穷，本题是 LL 范围。
注意到转移时 $O(1)$ 的，W 的范围最多不会超过 $[-2 \times 13 \times n, 2 \times 13 \times n]$ 因此算法复杂度是 $O(52n^2k)$。

Problem 5: 【ARC116E】Spread of Information

题意

给出一棵具有 n 个结点的树，现在国王想把一个信息传递给每一个结点上的人。

国王可以选择 K 个结点，让这些结点在时刻 0；立即获得这个消息。

如果一个结点在时刻 t 获得了消息，那么他会在让它的邻居在时刻 $t+1$ 知道这个消息。

请选择初始的 K 个点，使得尽快能让所有结点知道这条消息，请输出最早能让所有结点都知道该消息的时刻。

数据范围： $1 \leq K < n \leq 10^5$。

提示

考虑二分答案。

Problem 5：【ARC116E】Spread of Information

思路

二分答案 r，问题转变为，已知最大距离 r ，可以在树上选择 K 个结点作为哨兵，检验能否存在一种方案，使得任意结点距离自己最近的哨兵都不超过 r 条边。

考虑贪心过程：首先，找到深度最深的叶子。在这个叶子的 r 级父亲结点放置一个哨兵，一定是最优策略的一步。然后我们移除这个哨兵能在 r 步范围内扩散到所有结点。反复执行此过程，直到使用所有的 K 个哨兵，然后检查是否还存在未被覆盖的结点即可。

Problem 5: 【ARC116E】Spread of Information

实现

上述贪心过程的一种实现方法，是用dp形式维护。

定义 $dp(i)$ 表示考虑节点 i 子树中距离 i 最远的未被覆盖的结点与 i 的距离。特殊的，如果节点 i 的子树已经全部覆盖，则 $dp(i)$ 记录为溢出的覆盖距离，用负数表示。比如 $dp(i)=-(r-1)$ 则可以表示 i 的一个孩子放置了哨兵，该哨兵可以溢出 i 的子树从而再向 i 的父亲边往上溢出覆盖 $r-1$ 距离的结点。

转移的时候，考虑结点 u 求解 $dp(u)$，不妨设其孩子结点为 $v_1,v_2,\cdots,v_k$，则首先对其 dp 为负的孩子，找出最小值，假设为 $-x$ 那么意味着 u 结点还可以利用 $(x-1)$ 的覆盖距离去覆盖其他子树。然后考虑其 dp 值为正的孩子结点，找出最大值，不妨设其为 y 。

如果 $y+1\le x-1$ 说明 u 的子树可以填满了，直接有 $dp(u)=-(x-1)$ ；如果 $y+1>x-1$ 则说明 u 的子树还是填不满，于是有 $dp(u)=y+1$。如果此刻得出 $dp(u)=r$ 了，则说明此刻需要在 u 结点放置一个哨兵，于是 cnt 自增一次记录，然后令 $dp(u)=-r$。

总复杂度，二分+贪心+dp维护贪心状态，复杂度 $O(n\log n)$。

Problem 6: 【HDU7221】Darkmoon Faire

题意

给定一个长度为 n 的序列 $a_1, a_2, \cdots, a_n$，其中 a_i 互不相同。

你可以把序列划分成若干段，但是要求划分结果中，每一段的最大值位于奇数下标，最小值位于偶数下标。（下标指的是原来序列中这个值是第几个位置，不是划分后的段内下标）

求有多少种不同的划分方案。

$1 \leq n \leq 3 \times 10^5$。

样例

输入：

5

3 2 4 1 8

输出：

2

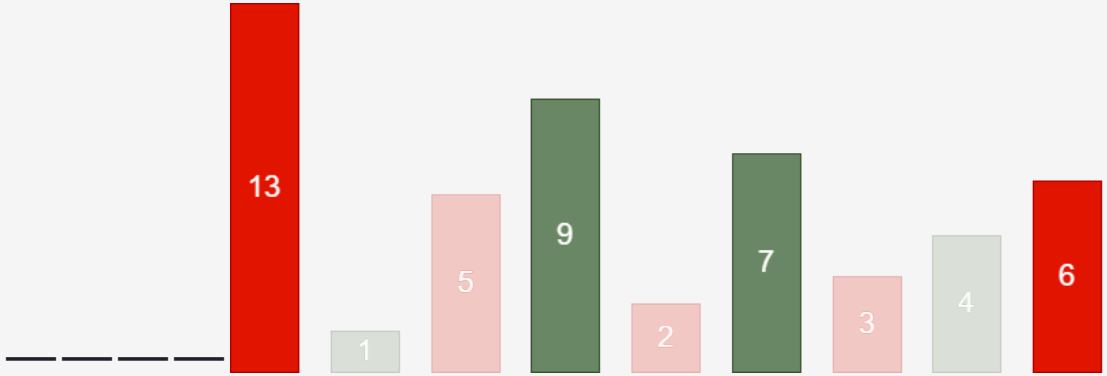
即有两种划分方法 $[3,2,4][1,8]$ 和 $[3,2][4,1,8]$。

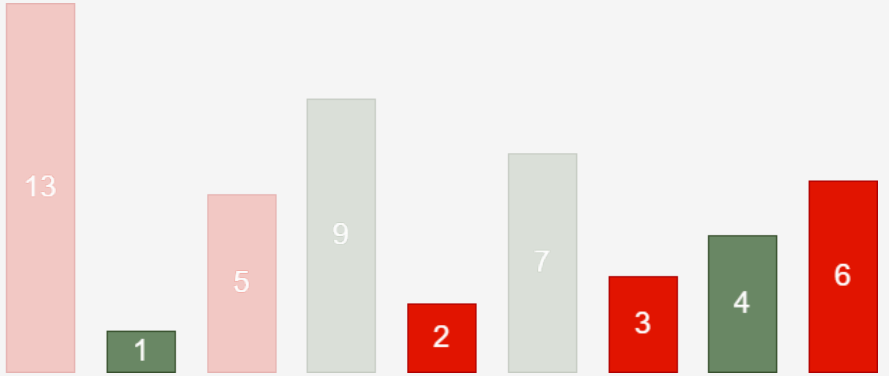
提示：和单调栈有关

Problem 6: 【HDU7221】Darkmoon Faire

思路

- 朴素的dp：$dp(i)$ 表示考虑序列 $a_1, a_2, \cdots, a_i$ 的合法划分方案数。转移的时候枚举 $1 \leq j < i$ ，满足区间 $[j+1, i]$ 的最值条件即可从 $dp(j)$ 转移累加到 $dp(i)$ 。复杂度 $O(n^2)$ 。





- 优化：观察扫描 i 的过程中，此时考虑的是区间右端点 $r = i$ 。观察维护单调栈的性质：以单调递减的单调栈为例，栈内每个元素都代表了一段 l 的范围（比如，如果栈内元素 a_x 底下的元素是 a_y，那么对于范围 $l \in [y+1, x]$ 时，数组区间 $[l, r]$ 内的最大值都是 a_x ），只要 l 在该范围内，最大值即为该元素。而对于单调递增的单调栈，则同理，蕴含了固定右端点时，l 变化时区间 $[l, r]$ 内的最小值。

Problem 6：【HDU7221】Darkmoon Faire

思路

我们发现，根据单调栈内元素的下标奇偶性，可以把序列分成若干段，称之为“奇段”和“偶段”。

递减栈中的每个段由“最大值”确定，因此递减栈的“奇段”作为左端点是必要的。同时，递增栈中的每个段由“最小值”确定，“偶段”作为左端点也是必要的。只有当一个位置 l 同时处于递减栈的“奇段”和递增栈的“偶段”时，才可以作为左端点，用 $dp(l-1)$ 累加到 $dp(r)$ （也就是累加到 $dp(i)$）。

实现

我们可以用线段树来维护每个位置在两个栈里所处的段的奇偶性，进而利用线段树快速求和。实现方式不唯一，以下是其中的一种。

在线段树每个结点维护四个值 $w(0,0),w(1,0),w(0,1),w(1,1)$，$0/1$ 表示的是在两个栈里所处的段的奇偶性，w 表示维护的对应的价值和。只有 $w(1,0)$ 的值是我们需要的。一开始 $w(0,0),w(0,1),w(1,1)$ 都为 0，表示这些情况没有贡献。

对于单调栈维护过程中，我们会删掉一些区间，并添加一个新区间这等价于区间覆盖。但我们发现，有两个栈上操作时，区间覆盖不容易实现；事实上，可以将覆盖等价于将一些删掉的区间的奇偶性取反，两个栈分别导致的取反操作在线段树上是可以通过两个tag维护的，并且两组交换操作满足交换律和结合率。实现上有一些细节，复杂度 $O(n\log n)$。